CS3213: Foundations of Software Engineering

White-box Testing

National Space Agency of Singapore (NSAS)

MTI MINISTRY OF TRADE AND INDUSTRY SINGAPORE

ESTABLISHMENT OF THE NATIONAL SPACE AGENCY OF SINGAPORE

The Government will set up the National Space Agency of Singapore (NSAS) from 1 April 2026. NSAS will be established under the aegis of the Ministry of Trade and Industry (MTI) to lead Singapore's national space ambitions to be a leader in space technologies and a credible contributor to the global space ecosystem.

https://isomer-user-content.by.gov.sg/166/dee00025-1a8a-4db3-a98b-aeb59512f66a/Press%20Release%20on%20Establishment%20of%20NSAS.pdf

National Space Agency of Singapore (NSAS)

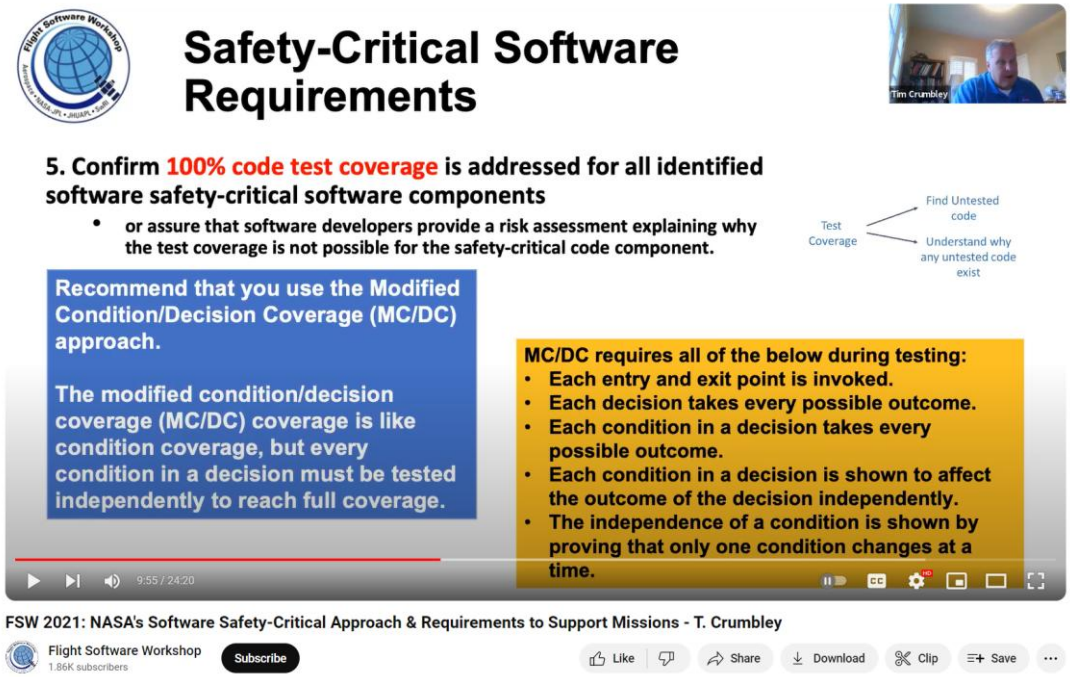
Image by Gemini

Ariane 5

The rocket self-destructing 37 seconds after launch because of a malfunction in the control software

Image by Gemini

MC/DC Coverage



https://www.youtube.com/watch?v=tE76V4pSjrY

MC/DC

Richard Hipp Speaks Out on SQLite

Marianne Winslett and Vanessa Braganho

Richard Hipp
http://www.hwaci.com/drh/index.html

"You say that SQLite has aviation grade testing. What does that mean?"

SQLite

White-box

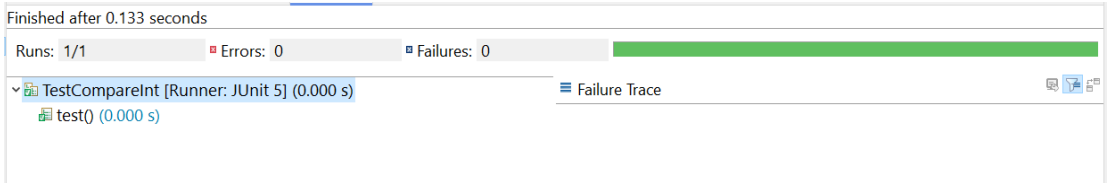
- **White-box testing**: use the source code to guide testing
- Alternative terms
  - Clear-box testing or glass-box testing
  - Also known as *structural testing*

White-box Testing

- Main means through *coverage criteria*
- Reasons
  - Potentially forgotten partition when analysing the requirements
  - Knowledge of requirements-independent implementation aspects (e.g., caching, specialized data structures, …)

Example: Optimization (1/2)

```
public static boolean integersAreEqual(Integer a, Integer b) {
    return a == b;
}
```




```
@Test
void test() {
    assertTrue(integersAreEqual(0, 0));
    assertTrue(integersAreEqual(1, 1));
    assertTrue(integersAreEqual(-1, -1));
    assertFalse(integersAreEqual(0, 1));
    assertFalse(integersAreEqual(-1, 0));
    assertFalse(integersAreEqual(-1, 1));
}
```

Example: Optimization (2/2)

valueOf

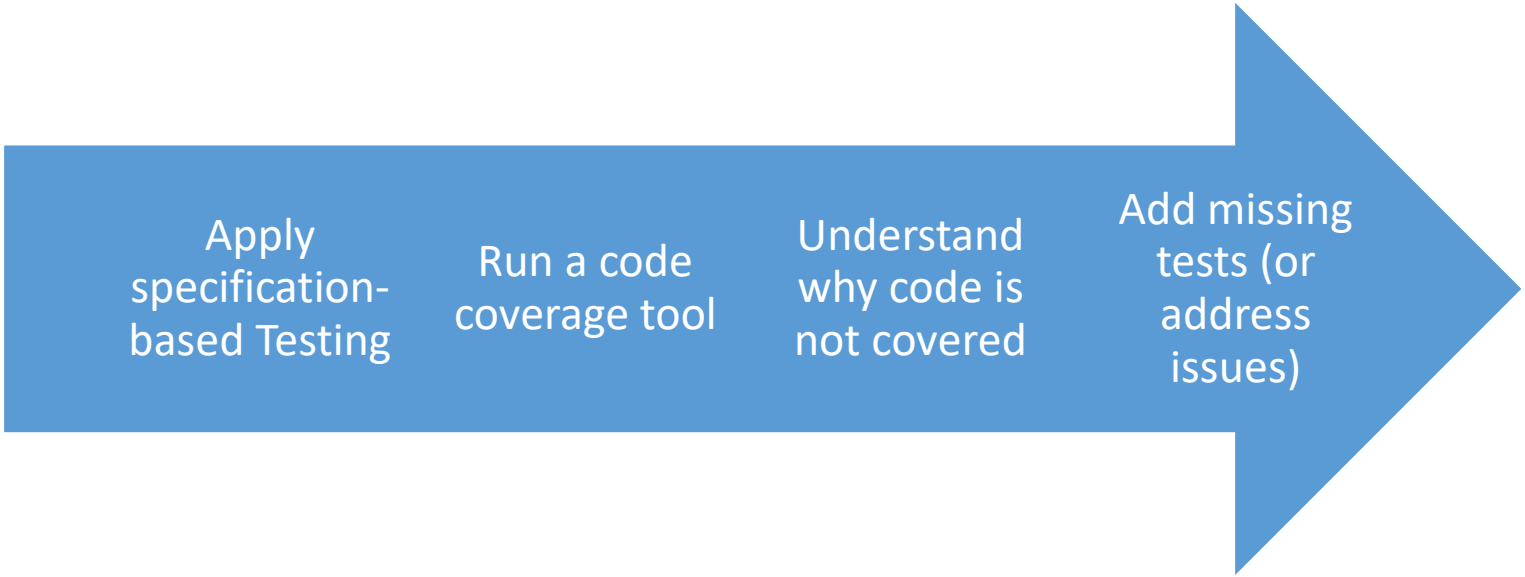
```
public static Integer valueOf(int i)
```

Returns an `Integer` instance representing the specified `int` value. If a new `Integer` instance is not required, this method should generally be used in preference to the constructor `Integer(int)`, as this method is likely to yield significantly better space and time performance by caching frequently requested values. This method will always cache values in the range -128 to 127, inclusive, and may cache other values outside of this range.

The implementation only works for integers within the range of -128 to 127 due to an internal optimization! Specification-based testing might have overlooked this case.

https://docs.oracle.com/en/java/javase/23/docs/api/java.base/java/lang/Integer.html

Applying Structural Testing



Coverage Criteria

- Line coverage
- Branch coverage
- Condition + branch coverage
- Path coverage
- MC/DC Coverage

Example

- Count the number of words in a string that end with either “s” or “r”
- A word ends when a non-letter appears
- The program returns the number of words

Example

```
public static int count(String str) {
    int nrWords = 0;
    char lastChar = 0;
    for (int i = 0; i < str.length(); i++) {
        char currentChar = str.charAt(i);
        if (!Character.isLetter(currentChar) &&
            (lastChar == 'r' || lastChar == 's')) {
            nrWords++;
        }
       lastChar = currentChar;
    }
    if (lastChar == 'r' || lastChar == 's') {
        nrWords++;
    }
    return nrWords;
}
```

Example

```
@Test
void twoWordsEndwingWithS() {
    assertEquals(2, CountWords.count("dogs cats"));
}


@Test
void noWords() {
    assertEquals(0, CountWords.count("dog cat"));
}
```

Jacoco

- Jacoco: the perhaps most popular code coverage tool for Java
- Based on Java bytecode instrumentation
- Shows only basic kinds of coverage (line coverage, highlights incompletely-covered branches)

https://www.jacoco.org/jacoco/

Full Line Coverage

coverage	100.0 %	52	0	52
CountWords.java	100.0 %	37	0	37
CountWords	100.0 %	37	0	37
count(String)	100.0 %	37	0	37
TestCountWords.java	100.0 %	15	0	15

```
 9    public static int count(String str) {
10        int nrWords = 0;
11        char lastChar = 0;
12        for (int i = 0; i < str.length(); i++) {
13            char currentChar = str.charAt(i);
14            if (!Character.isLetter(currentChar) && (lastChar == 'r' || lastChar == 's')) {
15                nrWords++;
16            }
17            lastChar = currentChar;
18        }
19        if (lastChar == 'r' || lastChar == 's') {
20            nrWords++;
21        }
22        return nrWords;
23    }
```

The *r* case is not covered

Full Branch Coverage (Bytecode Level)

```
@Test
void twoWordsEndwingWithR() {
    assertEquals(2, CountWords.count("dogr catr"));
}
```

```
public static int count(String str) {
    int nrWords = 0;
    char lastChar = 0;
    for (int i = 0; i < str.length(); i++) {
        char currentChar = str.charAt(i);
        if (!Character.isLetter(currentChar) && (lastChar == 'r' || lastChar == 's')) {
            nrWords++;
        }
        lastChar = currentChar;
    }
    if (lastChar == 'r' || lastChar == 's') {
        nrWords++;
    }
    return nrWords;
}
```

Coverage Criteria

- Line coverage
- Branch coverage
- Condition + branch coverage
- Path coverage
- MC/DC Coverage

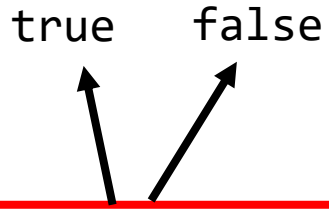
$$\frac{lines\ covered}{total\ lines} \text{ x } 100\%$$

Coverage Criteria

- Line coverage
- Branch coverage
- Condition + branch coverage
- Path coverage
- MC/DC Coverage

$$\frac{branches\ covered}{total\ branches} \text{ x } 100\%$$

Branch Coverage




```
if (!Character.isLetter(currentChar) && (lastChar == 'r' || lastChar == 's')) {
    nrWords++;
}
```

Coverage Criteria

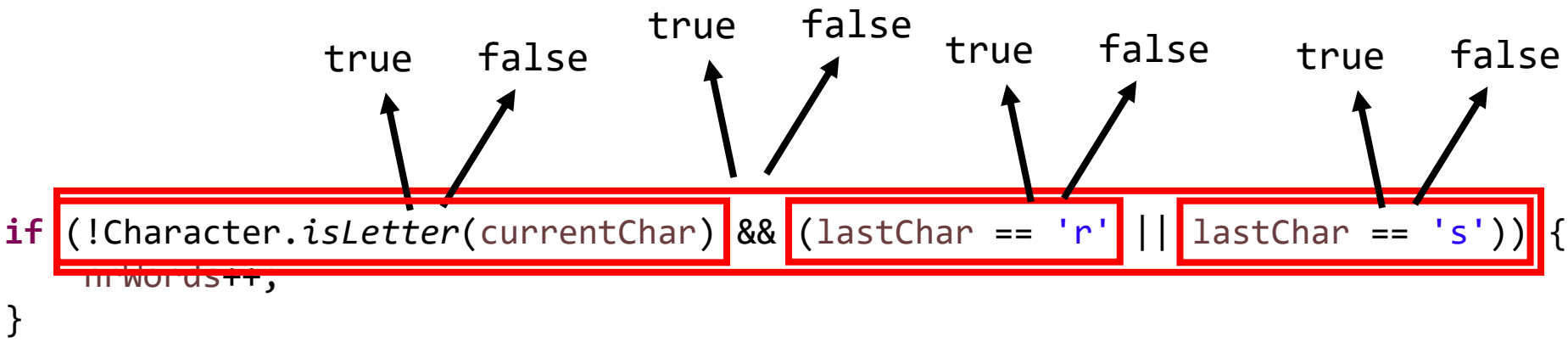
- Line coverage
- Branch coverage
- Condition + branch coverage
- Path coverage
- MC/DC Coverage

$$\frac{branches\ covered + conditions\ covered}{total\ branches + total\ conditions} \text{ x } 100\%$$

Condition + Branch Coverage

- In addition to consider branches, considers every *condition* of each branch statement
- Condition: A Boolean expression containing no Boolean operator
  - Example: a > 5
  - Counterexample: a > 5 AND a < b

Condition + Branch Coverage



Condition + Branch Coverage

Test case number	*isLetter*	== 'r'	== 's'	*isLetter && (== 'r' || == 's')*
1	True	True	True	True
2	False	False	False	False

Does condition coverage imply branch coverage?

Condition + Branch Coverage

- Does condition coverage imply branch coverage? No
- Consider: Predicate A || B used in a branch
  - T1: true and false
  - T2: false and true
  - The false-branch would be never executed

Coverage Criteria

- Line coverage
- Branch coverage
- Condition + branch coverage
- Path coverage
- MC/DC Coverage

$$\frac{paths\ covered}{total\ paths} \text{ x } 100\%$$

Path Coverage

- Often impossible or too expensive to achieve
- In a simple function with 10 conditions, the number would already be $2^{10} = 1024$
- Even more complicated with loops: need to be executed once, twice, three times, and so on

Can we have something stronger than Condition + Branch Coverage, but less expensive than Path Coverage?

MC/DC

Image by Gemini

Coverage Criteria

- Line coverage
- Branch coverage
- Condition + branch coverage
- Path coverage
- MC/DC Coverage

$$\frac{Number\ of\ conditions\ shown\ to\ independently\ affect\ the\ decision\ outcome}{Total\ number\ of\ conditions\ within\ the\ decisions} \times 100\%$$

MC/DC Coverage

- Modified Condition/Decision Coverage
- Ensures that each condition is tested in a way that proves it can independently affect the decision outcome
- Test important combinations of conditions and thus reduce the cost as compared to full Path Coverage
- Typically requires n + 1 test cases
  - n is the number of conditions

MC/DC Coverage

- Each decision takes every possible outcome
  - Decision: Boolean expression consisting of conditions and zero or more operators
    - In contrast to branches, also covers assignments to Boolean variables
- Each condition in a decision takes every possible outcome
- Each condition in a decision is shown to independently affect the outcome of the decision

MC/DC Coverage

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

MC/DC Coverage

Test case number	*isLetter*	== 's'		
1	True	Tru		
2	True	Tru		
3	True	Fals		
4	True	Fals		
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

- Each condition in a decision takes every possible outcome
- Each condition in a decision is shown to independently affect the outcome of the decision

MC/DC Coverage

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True			
4	True			
5	False			
6	False			
7	False	False	True	False
8	False	False	False	False

Can we find a test case where isLetter is false, the decision False, and the other conditions are unchanged?

MC/DC Coverage

Potential tests:
A: {T1, T5}

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

MC/DC Coverage

Potential tests:
A: {T1, T5}, {T2, T6}

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

If we continue looking, we discover additional possibilities

MC/DC Coverage

Potential tests:
A: {T1, T5}, {T2, T6} , {T3, T7}

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

If we continue looking, we discover additional possibilities

MC/DC Coverage

Potential tests:
A: {T1, T5}, {T2, T6} , {T3, T7}

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

Does not affect the decision

MC/DC Coverage

Potential tests:
A: {T1, T5}, {T2, T6} , {T3, T7}

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

Let us continue with B

MC/DC Coverage

Potential tests:
A: {T1, T5}, {T2, T6} , {T3, T7}

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

Decision unaffected

MC/DC Coverage

Potential tests:
A: {T1, T5}, {T2, T6} , {T3, T7}
B: {T2, T4}

Test case number	`isLetter`	`== 's'`	`== 'r'`	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

No additional pairs

MC/DC Coverage

Potential tests:
A: {T1, T5}, {T2, T6} , {T3, T7}
B: {T2, T4}

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

Can you identify the relevant test(s)?

MC/DC Coverage

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

Potential tests:

A: {T1, T5}, {T2, T6} , {T3, T7}

B: {T2, T4}

C: {T3, T4}

MC/DC Coverage

Test case number	`isLetter`	`== 's'`	`== 'r'`	`decision`
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

Potential tests:
A: {T1, T5}, {T2, T6} , {T3, T7}
B: {T2, T4}
C: {T3, T4}

We need one out of {T1, T2, T3} and one out of {T4, T5, T6, T7, T8}

MC/DC Coverage

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

Potential tests:
A: {T1, T5}, {T2, T6} , {T3, T7}
B: {T2, T4}
C: {T3, T4}

{T2, T3, T4} are definitely part of the final test cases, since they are the only ones covering B and C

MC/DC Coverage

Test case number	*isLetter*	== 's'	== 'r'	*decision*
1	True	True	True	True
2	True	True	False	True
3	True	False	True	True
4	True	False	False	False
5	False	True	True	False
6	False	True	False	False
7	False	False	True	False
8	False	False	False	False

Potential tests:

A: {T1, T5}, {T2, T6} , {T3, T7}

B: {T2, T4}

C: {T3, T4}

We do not want to add two additional tests, so the choice is between adding T6 and T7

MC/DC Coverage

- Final set of tests: {T2, T3, T4, T6} or {T2, T3, T4, T7}
- Cheaper than the eight tests we would need for path coverage

MC/DC

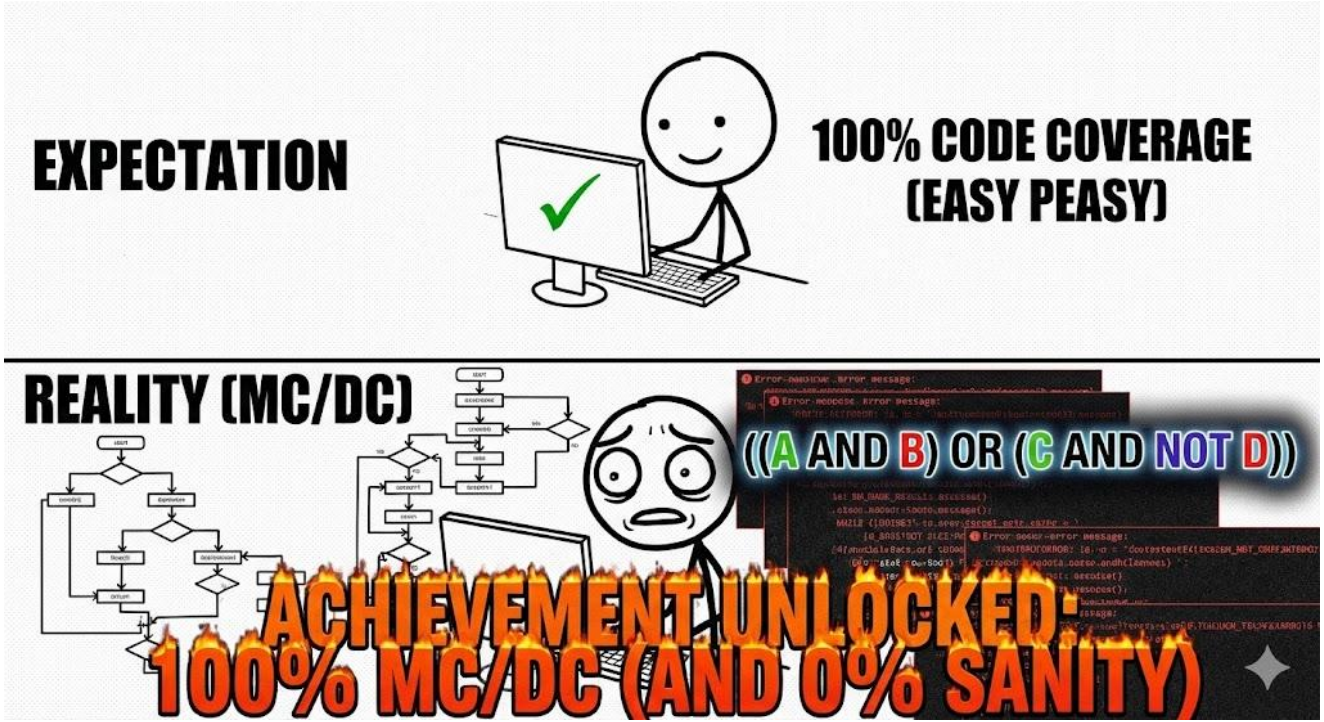
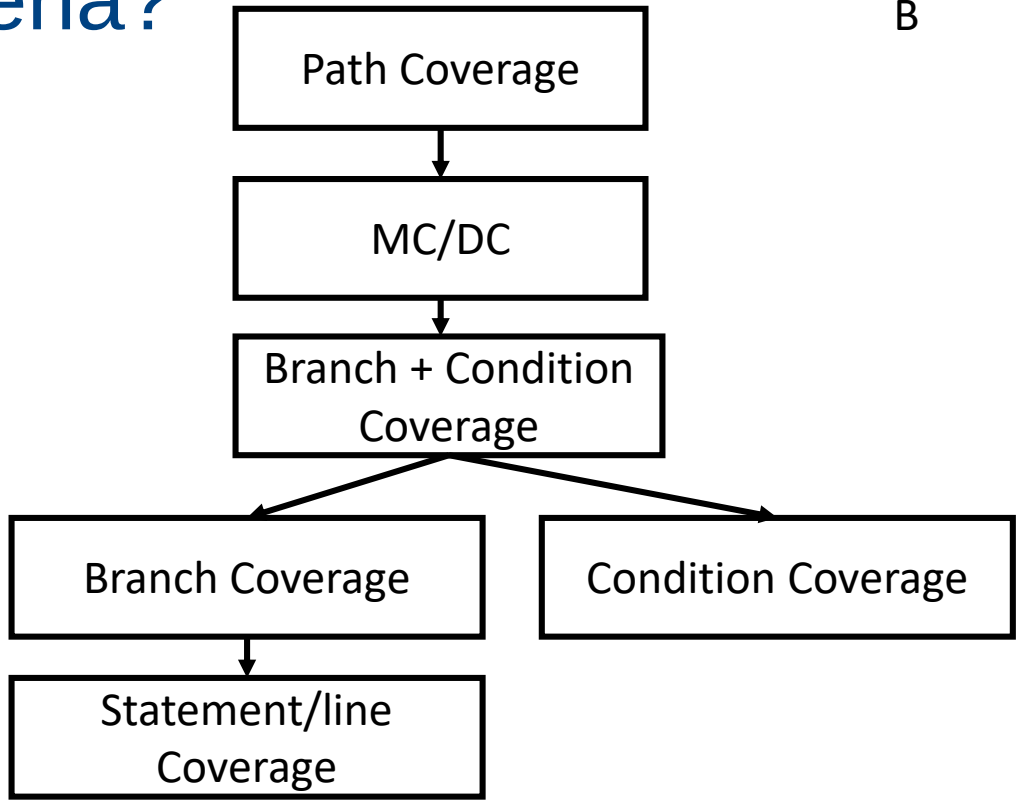


Image by Gemini

Which Criteria?

A
↓
B
A subsumes B



Code Coverage: Criticism

- 100% coverage can be achieved without testing anything (e.g., no assertions)

Image by Gemini

Code Coverage: Criticism

- Code coverage should be seen as a tool that helps you identify uncovered code
  - Should not be implemented blindly

MC/DC: a Silver Bullet?

- SQLite achieves 100% MC/DC coverage
- We have found more than 200 bugs in it

Richard Hipp Speaks Out on SQLite

Marianne Winslett and Vanessa Braganholo

Richard Hipp
http://www.hwaci.com/drh/index.html

"You say that SQLite has aviation grade testing. What does that mean?"

Summary and Takeaways

- Structural testing: why and when
- Line coverage
- Branch coverage
- Condition + branch coverage
- Path coverage
- MC/DC Coverage

Sources and References